Relatório de Desenvolvimento e Testes da API – Atividade de Carlos

Alunos: Aldo Filho - Programador, Murilo Senna e Davi Meneses - Testes de API, João Guilherme - Relatório, Milena - Apresentação no Canva

1. Estrutura e desenvolvimento do sistema

Inicialmente, o código foi desenvolvido utilizando a arquitetura MVC (Model, View e Controller), com as tecnologias Java e Spring Boot no backend e PostgreSQL como sistema de gerenciamento de banco de dados.

Para a criação e manutenção da estrutura do banco de dados, foi utilizada a abordagem de migrations, por meio da dependência Flyway. Essa ferramenta permite versionar e controlar as alterações realizadas no banco de dados, garantindo que as migrations sejam executadas de maneira organizada e sequencial.

De forma complementar, o arquivo application.yaml foi configurado para estabelecer a conexão entre a aplicação e o banco de dados, definindo parâmetros necessários para seu funcionamento. Essa configuração permite que a aplicação utilize a estrutura previamente estabelecida pelas migrations, mantendo a compatibilidade entre o código e a base de dados. Dessa forma, caso alguma informação ou estrutura esteja incompatível com o esperado, poderão ocorrer erros durante a inicialização ou execução do sistema.

Além disso, foi desenvolvido um Controller para organizar as requisições da API e estabelecer um padrão de acesso aos recursos do sistema. Essa organização utiliza endpoints, que definem os caminhos pelos quais o frontend consegue acessar e executar operações no backend.

Como exemplo, foram estabelecidas rotas como:

- http://localhost:8080/product
- http://localhost:8080/product/2

Esses endpoints permitem localizar os recursos e determinar quais operações podem ser realizadas, como cadastrar, consultar, atualizar e excluir produtos. Assim, cada funcionalidade do frontend possui uma rota correspondente para se comunicar com a API.

2. Relatório de testes da API

Após a implementação da estrutura inicial, foram realizados testes na API para verificar o funcionamento das operações de cadastro, consulta, atualização e exclusão de produtos, utilizando os métodos HTTP POST, GET, PATCH e DELETE.

2.1. POST – Registrar produto

Inicialmente, foi realizado um teste para registrar um novo produto na API.

- **Primeiro teste:** A requisição foi realizada com sucesso, retornando 201 Created.
- **Teste com nome em branco:** Foi realizada uma tentativa de cadastro com o campo nome vazio, resultando em 400 Bad Request. Durante a análise, identificamos que essa validação não estava devidamente tratada no código, existindo apenas a validação para valores nulos.

2.2. GET – Buscar produtos

Foram realizados testes para verificar a consulta de produtos por diferentes critérios.

- **Busca geral:** A consulta de todos os produtos retornou 200 OK.

- **Filtro por atividade:** Foi utilizado um filtro para buscar produtos ativos e inativos. A consulta dos produtos inativos também retornou 200 OK.
- **Filtro por ID:** Inicialmente, a consulta apresentava um erro devido à existência de um atributo adicional que não deveria estar presente. Após sua remoção, o teste retornou 200 OK.
- **Filtros combinados:** Foram combinados os filtros de ID e atividade. Ao buscar o produto de ID 3 com o campo de atividade definido como falso (inativo), a requisição retornou 200 OK.

2.3. PATCH – Atualizar produto

Foram realizados testes para verificar a atualização parcial dos dados de produtos.

- **Atualização de dados:** Foram atualizados o nome, o preço e a quantidade em estoque de um produto.
- **Nome em branco:** Ao tentar atualizar o nome utilizando um valor vazio, a requisição retornou 200 OK, permitindo que o produto permanecesse com o nome em branco.
- **Descrição em branco:** Ao atualizar a descrição com um valor vazio, também foi retornado 200 OK, deixando a descrição em branco.
- **Preço e quantidade em estoque:** Diferentemente dos campos anteriores, esses atributos não aceitaram atualizações com valores em branco.
- **Atualização da atividade:** Foi realizada a alteração da situação dos produtos de ID 3 e 4 para inativos.

2.4. DELETE – Excluir produto

Foi realizado um teste para verificar a exclusão de produtos.

- **Teste de exclusão:** A requisição retornou 204 No Content, indicando que a operação foi executada com sucesso, sem a necessidade de retornar conteúdo no corpo da resposta.

3. Conclusão

O desenvolvimento da aplicação permitiu estabelecer uma estrutura organizada utilizando a arquitetura MVC, com integração entre Java, Spring Boot e PostgreSQL. A utilização do Flyway também contribuiu para o controle e versionamento da estrutura do banco de dados, enquanto os Controllers e endpoints estabeleceram um padrão de comunicação entre o frontend e o backend.

Os testes realizados na API permitiram identificar e corrigir problemas relacionados ao cadastro de produtos e aos filtros de consulta. Após os ajustes, as operações apresentaram os códigos de resposta HTTP esperados.

Entretanto, durante os testes de atualização com o método PATCH, foi identificado que os campos nome e descrição ainda permitem o envio de valores vazios, possibilitando a gravação de informações incompletas no banco de dados. Portanto, essa validação permanece como um ponto de melhoria para garantir maior integridade e consistência dos dados armazenados.